

Note to the agent running the OptionScope walk-forward study

You are picking up two scaffolded files — `runner/walk-forward.ts` and `runner/handback-emitter.ts` — written **without repo access**. They are deliberately incomplete. Everything below is a thing you must correct, verify, or repair against the real code and real data before any result is trustworthy. Do not treat a clean compile as a finished study. The scaffolding is built to fail loud, not to be right.

Work in this order. Do not skip the reconciliation steps to get to a number faster — that is the exact mistake the trend strategy made (written first, tested afterward, burned 452 backtested trades before anyone checked).

1. Wire in-sample expectancy — BLOCKING, do this first

`buildAggregate` in `walk-forward.ts` sets `inSampleExpectancy = null`. This is on purpose. Until you replace it with the real number **computed by the harness over each run's own fit windows**, the emitter's integrity gate forces every verdict to INCONCLUSIVE, and it is right to.

Do NOT reuse the `+$2.61` in-sample figure from NCI_MEMORY 7c. That was the old 2-fold run. This 6+ fold run over a 2020-extended range must compute its own in-sample expectancy over its own fit windows. Find where the replay harness reports fit-window expectancy (or add that reporting) and pass it through. If the harness only reports OOS, you have to build the in-sample pass — without it there is no `edge_retained_pct` and no SURVIVED bar to clear.

2. Resolve whether scoreMeanReversion is fitted or fixed — this IS the study

The trend strategy died because the optimizer chose different params every fold (trailing/5, trailing/5, ladder/4, ladder/5). That instability was the signature of fitting noise.

Go read the walk-forward loop and `scoreMeanReversion()` and answer one question: **does anything get optimized over the fit window, or is the strategy fixed-parameter?**

- If it optimizes: capture the chosen parameters per fold and put them in the output. Parameter drift across folds is a finding on its own — arguably the most important one, because it's what killed the last strategy. The current scaffold does NOT capture this yet. Add it.
- If it's fixed-parameter: the fit window is only warmup/context. Say so explicitly in `deviations_from_spec`, because a reader will assume "walk-forward" means "refit each fold" and will misread the result otherwise.

3. Fix every `TODO(verify against repo)` marker

Search both files for `TODO(verify against repo)`. Each is a place I assumed a signature I couldn't see. The load-bearing ones:

- `runReplay` signature — I assumed `{barsBySymbol, window, scoreFn, exitPlan, config}` returning `TradeResult[]`. If the only entry today is the CLI behind `REPLAY.bat`, expose a callable it wraps and import THAT. Do not reimplement the replay loop — you must use the existing `visibleBars()` no-lookahead path (Ground Rule #1). Remove the `as any` once it's typed; if the `as any` is still there at run time, no-lookahead is unverified.
- `Bar` shape — match `data/bars-daily.json` exactly. If bars aren't calendar-aligned across symbols, the regime labeler's per-day averaging is wrong; align by date first (there's a TODO on that line).
- `TradeResult` shape — the CSV in the emitter only serializes 5 columns. A spot-check against a lossy CSV is worthless. Emit every column the harness produces.
- `loadConfig` / `loadBars` — confirm names and that `loadConfig` is genuinely read-only (Ground Rule #6: never mutate config risk values).

4. Extend the bar cache FIRST, and reconcile survivorship

`loadBars` assumes `data/bars-daily.json` already reaches back to $\geq 2020-01-01$. Right now the cache is 2024-01-02 $\rightarrow$ 2026-08-14 (NCI_MEMORY 7b). **Extend it before running anything**, via the same MCP pull method that generated the existing file — find the script/command that produced `bars-daily.json` and reuse it with the wider date range.

Then trust the reconciliation output. `reconcileUniverse` will list symbols that returned no bars. **For each missing symbol, determine by hand whether it's a data gap or a real delisting/acquisition.** The MCP only serves currently-listed names; a symbol that died in 2021 will silently vanish and reintroduce survivorship bias that no error will catch. This is the trap that most threatens Ground Rule #3. The `partial` list (bars starting late) matters too — those names are absent from early folds, so your 2020/2021 folds are effectively a smaller universe than your 2025 folds.

5. Things the scaffold decided that you should re-decide with real data

- **Regime thresholds** (`walk-forward.ts`): $\pm 10\%$ endpoint return, forced-bear at 20% intra-window drawdown. The drawdown override exists to catch the COVID V-shape (start $\sim$ end, crash in the middle). Sanity-check the labels the real data produces — if a fold you know was choppy comes back "bear", the threshold is wrong.
- `THIN_BEAR_FOLD_TRADES = 20`: below this, a bear fold is treated as too thin to trust and the verdict is forced INCONCLUSIVE even if aggregate math clears the SURVIVED bar. This encodes the handoff's warning against thin bear samples. If your trade frequency makes 20 the wrong cutoff, change it — but do not remove the check. "6 folds but the bear windows were empty" must not read as SURVIVED.
- **COVID fold expectation**: pre-registered as likely INCONCLUSIVE for a buy-fear strategy (it buys all the way down a vertical crash; ATR-capped 5% stops + gap-at-open fills will either

look catastrophic or, on the V-recovery, misleadingly great — both are regime artifacts, not edge). Lean on the 2022 grind fold for the real bear read; it's the better analog for a growth/momentum universe anyway.

6. Anomaly log is seeded with a placeholder — replace it

`bugs_or_anomalies_found` starts with a PLACEHOLDER string. An empty array must mean "checked, found none," never "didn't check." The prior study found 3 real harness bugs (stopped clock, wrong sequencing, halt counters never resetting) precisely by looking. Look. Then replace the placeholder with real findings or an explicit "reviewed X, found none."

7. NOTE-A — schema shape to confirm with NCI

Section 5 shows `test1_walk_forward` as a single object. The scaffold emits an **array of two** (ladder + fixed) because the handoff asks the test to run both. Confirm NCI's ingester accepts the array; if it wants one object with a nested `exit_plans` map instead, reshape in the emitter — the data's the same, only the container changes.

8. What NOT to do

- Do not place any broker order, live or paper. Do not touch `runner/gates.ts`-gated paths or `runner/config.ts` risk values (Ground Rules #5, #6).
- Do not soften a bad verdict. INVALIDATED or INCONCLUSIVE is an expected, useful outcome. The scaffold's gate can only make verdicts more conservative, never less — keep it that way.
- Do not use `scoreBullish` for anything except an explicit negative control. It is dead (−\$2.40 OOS, sign-flipped).
- Do not fill `repo_commit_sha` with anything but the real hash you ran against. The emitter blocks a missing/invalid one; don't defeat that check.

9. Test 2 sweep (`runner/param-sweep.ts`) — what it needs from you

The sweep harness cannot be run until you supply, from the actual repo, the pieces it deliberately does not guess:

- **The `SweepParam` objects.** You said `scoreMeanReversion` has multiple params and weren't sure of the names. So the harness takes them as input. For each param you want to sweep, build a `SweepParam` with: the real `name`, the `defaultValue` currently in the codebase, a `grid` of **≥5 values bracketing that default** (handoff Section 4), and an `inject(base, value)` function that returns a **fresh** config/params object with the value applied. `inject` **MUST NOT** mutate `base` — Ground Rule #6, and a mutation would corrupt every subsequent grid point. Write `inject` yourself because only you can see where each param actually lives (strategy params vs exit-ladder params vs config).

- **Sweep one param at a time, others at default.** The harness already enforces this structure. Do not "improve" it into a joint grid — a joint sweep lets a good value on one knob mask a fragile value on another, which is the exact confound that would hide a lucky corner. The handoff asks for entry swept under the ladder, then exit-ladder params swept separately with entry held at default. That's what the code does; keep it.
- **`buildFolds` must be shared, not duplicated.** `param-sweep.ts` imports it from `walk-forward.ts` (currently private — export it, or better, lift folds into a shared module both import). If you copy-paste the fold logic instead, the two copies WILL drift and your Test 1 and Test 2 folds won't be comparable, which quietly voids the whole point of running the sweep on the same scheme.
- **Default to `mode: "walk_forward"`, not the degraded fallback.** The handoff allows a single-window fallback "if full walk-forward per grid point is too expensive." Only reach for it if a full run is genuinely infeasible on your hardware — measure first. Fragility is an out-of-sample property; a single-window sweep that says "stable" is weak evidence and the harness will stamp it `[DEGRADED]` in the output so it can't be mistaken for the real thing. If you can afford walk-forward per grid point ($\approx 120\text{--}200$ replay runs for a typical grid), do that.
- **`final_equity_usd` needs the real equity-floor logic.** Right now `pointMetrics` does a naive `startingEquity + total`. The prior study's blown grid points read as $\sim \$200$ because they hit the config equity floor (`minScore-5` $\rightarrow$ $\$196$). Wire the real floor/starting-equity logic from `config/gates` so your final-equity column is comparable to NCI_MEMORY 7b/7c. Also pull the real starting equity ($\$300$) from config rather than passing a literal.
- **Confirm the two `TODO(confirm)` decisions:** (a) sweep entry params under BOTH ladder and fixed, or just ladder? The code does just the plan you pass; the handoff swept under ladder, but running both is cheap insurance. (b) Schema shape when there are multiple entry params — the code emits the first as `entry_threshold_sweep` and the rest under `additional_entry_sweeps`. Confirm NCI's ingester accepts that, or reshape (mirror of NOTE-A).
- **Read the `stability_assessment` strings the harness produces, don't just pass them up.** The assessment flags the specific "default sits adjacent to a sign flip" pattern that killed the trend strategy. If it fires FRAGILE, that is a real result — report it as plainly as the prior study said the trend strategy was DEAD. Do not average it away or bury it under the aggregate.

10. Test 3 wider-universe (`runner/wider-universe.ts`) — the survivorship test

Read the header comment in that file before touching it. The short version: **Test 3 is the most survivorship-dangerous of the three, and its reconciliation step is the actual test, not boilerplate.** You are asked to construct a 50-100 name list that deliberately includes names that "fell hard." The moment you pull bars for a 2020-era list of fallen names, every name that actually *died* — delisted, acquired, went bankrupt — returns nothing from the MCP and silently drops out, leaving you with a survivors-only universe that biases the result *upward*. The wider you go, the

worse it gets, because the fallen names you included on purpose are the ones most likely to be gone.

Concretely:

- Build the intended 50-100 list and pass it as `intendedWiderUniverse`. Include the disasters (the handoff names CHPT -97%, LCID -90%, PLUG -81%, NIO, SNAP as the spirit to match) — do not quietly build a large-cap list.
- After the run, `reconciliation.missing` lists every name that returned no bars. **Check each one by hand.** A missing name is either a benign data gap or a real death. If it's a real death, either source its bars from before delisting (so the loss is in-sample) or, if you can't, record explicitly that the universe is survivorship-biased and the result reads optimistic. Do not just proceed.
- **Wire `test1ComparisonExpectancy` from the extended Test 1 aggregate** on the same exit plan (ladder). Do NOT substitute the old 2-fold +\$1.23 figure from NCI_MEMORY 7c — the extended Test 1 has its own number, and comparing against the stale one produces a wrong "holds/strengthens/weakens" verdict. If you pass null, the comparison string says so instead of lying.
- The file flags a **single-regime carry**: if the wider-universe edge comes entirely from bull folds while bear/choppy are flat, "holds on a wider universe" overstates it. Read `_regime_breakdown` and report honestly if the edge is regime-bound.
- Same shared-export requirement as Test 2: `buildFolds`, `labelRegime`, `reconcileUniverse`, `summarizeTrades` must be exported from `walk-forward.ts` (or lifted to a shared module) and reused, never duplicated. If Test 3 uses a different fold/label definition than Test 1, the comparison the whole test exists to make is void.
- Test 3 returns `_caveats` and `_all_trades_for_csv`. Merge `_caveats` into the file-level `caveats` array when you assemble the emitter input, and write a `test3_universe.csv` from the trades if you want NCI to spot-check it (recommended — Test 3 is the one most likely to be quietly wrong).

11. Sequencing reminder for the whole handback

Test 1 tells you whether the **equities** mean-reversion path is arguable. It says nothing about the options contradiction (NCI_MEMORY 7d / next-action #7): the proven edge is short premium, and the agentic MCP only places long single-leg options. A SURVIVED Test 1 does NOT grant permission on the options side. Keep those two conclusions separate in whatever you report upward, or a good equities result will get misread as a green light on the exact trade type that lost \$49k.